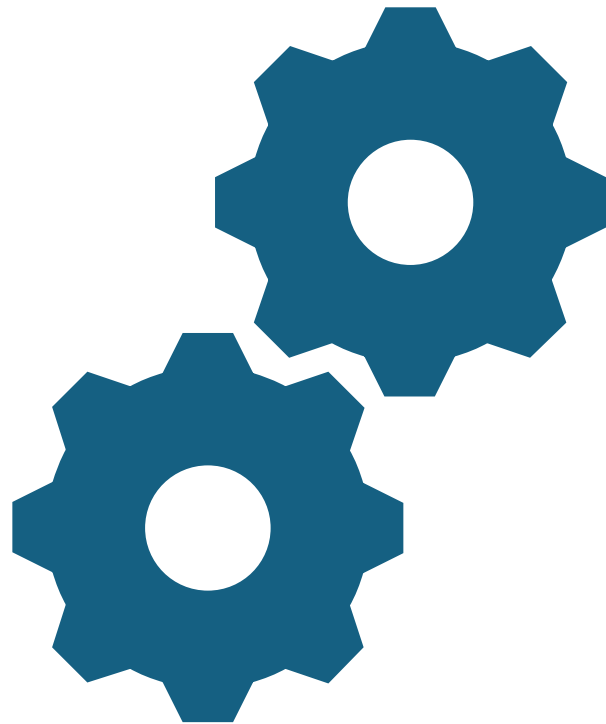




Jarrett Lane and James Young

Intro to Reverse Engineering

What are executables/binaries?

-
- These are what we commonly reverse engineer in CTFs
 - Binaries are files containing instructions for hardware to run something
 - Executables are made from compiled code, this means human readable code is "compiled"/translated into machine readable instructions
 - If you want to get human code from an executable, you need to decompile the executable, we use tools to help us do this
 - There is also assembly, which is a more readable way of representing pure machine code/instructions

What can you find in an executable?

-
- The code itself
 - Imported code from other libraries (static)
 - References to code to other libraries (dynamic)
 - Strings, if any text
 - Variables
 - Metadata
 - Random blankness that doesn't matter

Tools we use

-
- The three common programs we use for reverse engineering are Binary Ninja, Ghidra, and Ida
 - There is also a web tool for quick and easy things called dogbolt.org
 - We will use binary ninja because it's a bit more beginner friendly
 - Lets look at Binary Ninja together

Techniques for reversing

Especially in CTF

Rename parts of the code

-
- When compiling and decompiling, names of things like variables and functions often lose their original name
 - Ex: 'int count' might look like 'int rax_0934' when decompiling
 - If you can piece together what a function/variable does, try BurpSuites feature of renaming it, this will help you navigate the binary and make reading the decompiled code so much easier

Know where to look

-
- Start at the main function, and see if it calls other functions, takes input, or references any interesting data
 - Look at the listed symbols ("entities" or things in a binary) and see if you can find any interesting functions, variables, data, etc.
 - Look for any references to any type of flag
 - Look at any strings
 - Anything generally anomalous

Obfuscation

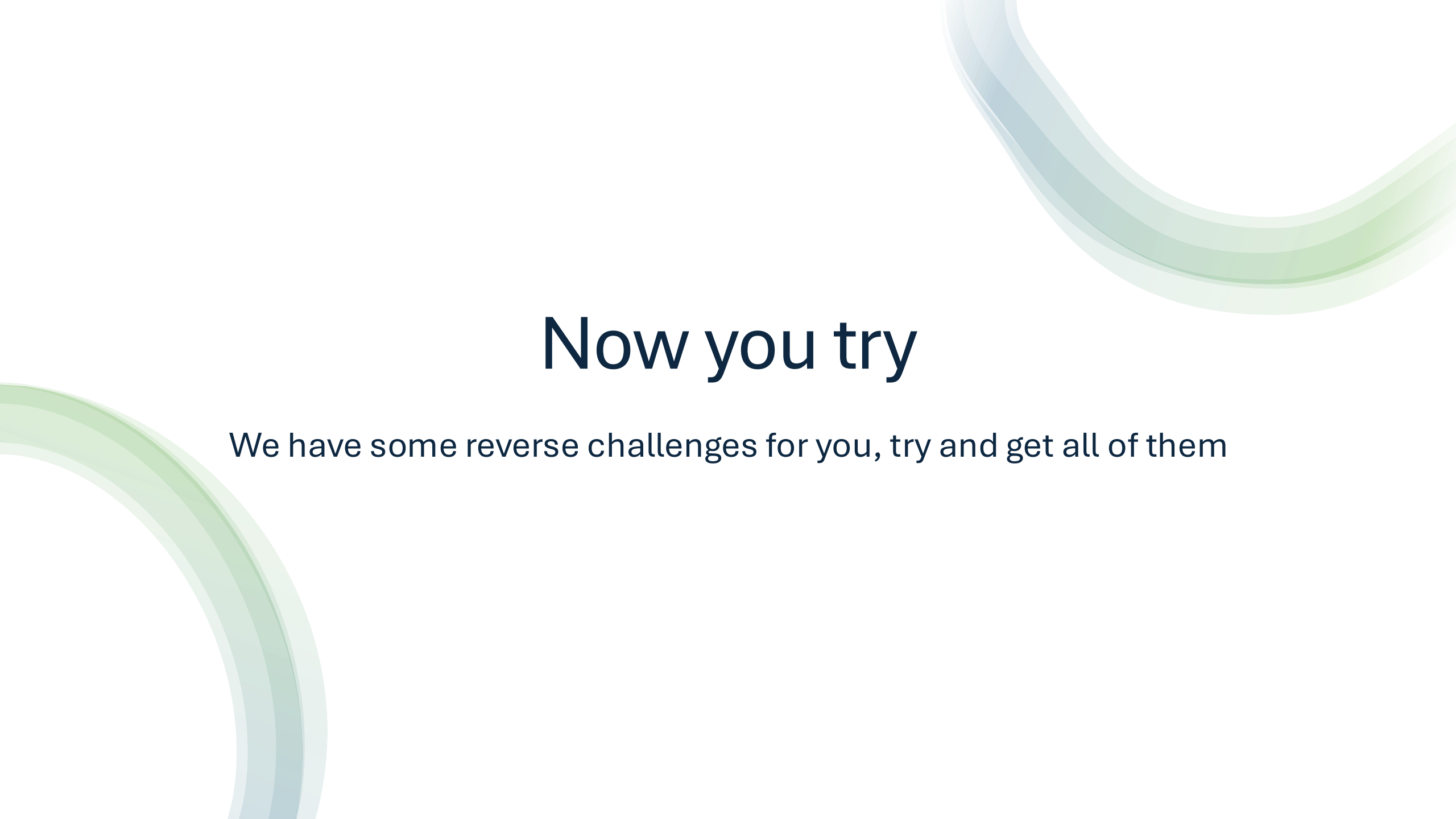
- Common obfuscation techniques are:
 - String encryption/encoding and runtime decryption
 - Packing and binary compression
 - Junk code and dead code
 - Anti-debugging and anti-disassembly tricks
- How to get around these:
 - Use dynamic reversing (debuggers)
 - Patch anti-debug checks and unpack binaries
 - See if they used an automatic obfuscation tool
 - Feed it to AI to deobfuscate it
 - Cope

Use AI

-
- AI can help you get a general idea of what the decompiled code is doing, it can also help explain tough areas like assembly
 - This is great for beginners with little C experience
 - This does not mean rely on AI to tell you everything

Debugging/ Dynamic reversing

-
- In some challenges, reversing can mean actually watching it run, not just examining code
 - Using a debugger lets you see the complete state that a program is in at a certain point of execution
 - This can tell you things like what a variable is, what is loaded in memory, etc
 - This is another topic that could be a meeting itself



Now you try

We have some reverse challenges for you, try and get all of them



Resources

- <https://www.youtube.com/watch?v=75gBFiFtAb8>
-